Real-world (build and search a real knowledge base) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Same idea, naming the exact pieces:

- `client.embeddings.create(model="text-embedding-3-small", input=text)` — turns one file's text into a vector.
- `chromadb.Client()` — a local vector database, no server to set up.
- `chroma_client.create_collection(name="knowledge_base")` — a named table of documents plus their embeddings.
- `collection.add(documents=[...], embeddings=[...], ids=[...])` — three parallel lists: raw text, matching embedding, and a unique id (the filename works).
- `collection.query(query_embeddings=[question_embedding], n_results=3)` — note `query_embeddings`, not `query_texts`, since you embedded the question yourself.

Because you wrote every document, you're your own ground truth — you can check each search result by eye and know immediately whether it's right or wrong, no guessing required.

Two things break the moment this stops being a script you run exactly once.

First: `chroma_client.create_collection(name="knowledge_base")` raises an error if a collection named `"knowledge_base"` already exists. Run your script a second time — to add a new document, or just because you re-ran it while testing — and it crashes on line 2, before doing anything useful. A script, a test file, and a notebook might all build "the knowledge_base collection" the same way this document's Build Task expects several files to share a config loader — and none of them should care whether it's the first call or the fiftieth. `chroma_client.get_or_create_collection(name="knowledge_base")` fixes this: it returns the

existing collection if one's already there, and creates it only if it isn't — the same "check first, only do the real work if needed" idea from Doc01's Build Task cache.

Second: ask an off-topic question — something none of your documents are actually about — and `collection.query()` still confidently returns your `n_results` closest matches, because that's what a nearest-neighbor search always does: it finds the closest *available* things, even when none of them are actually close. Nothing in the code signals "none of these are good matches." A real system checks the returned distance/score against a cutoff, and either drops the weak results or tells the caller explicitly that nothing relevant was found — instead of quietly handing the model 3 irrelevant chunks and letting it guess from there.

The extra pieces:

- `get_or_create_collection(name="knowledge_base")` instead of `create_collection(name="knowledge_base")`.
- A `SCORE_CUTOFF` constant, and a filter step after `collection.query()` that drops any result whose distance is worse than the cutoff, before returning. The distance is lower for closer matches; with OpenAI embeddings it is `2 - 2 × cosine similarity`, so start around `1.2` and tune it.
- `Path(folder).glob("*")` and `path.suffix` — the `pathlib` way to keep only `.txt` / `.md` files. Embed all of them in one call (`input=` a list), and pass four lists of the same order to one `collection.add(ids=..., embeddings=..., documents=..., metadatas=...)`.
- `chroma_client.delete_collection(name=...)` then `create_collection(...)` — a clean rebuild, so an **edited** document is picked up (`add()` with an existing id is silently ignored). On the first run there's nothing to delete: catch `chromadb.errors.NotFoundError`.
- LangChain's `Chroma(collection_name="knowledge_base", embedding_function=OpenAIEmbeddings(model=...), persist_directory="./chroma_db")` keeps the store on disk; `store.delete_collection()` empties it; `store.similarity_search_with_score(question, k=3)` returns `(Document, score)` pairs.

Sketch what a `search(question, cutoff)` function's signature and return type should look like once it can return "nothing relevant" as a real, distinct case — not just an empty list that looks the same as "no documents at all."

Hint 2 — The plan, and almost the whole thing

```
define:
    def build_vector_store(doc_folder: str):
        client = chromadb.Client()
        collection = client.create_collection(name="knowledge_base")
        for each file_path in Path(doc_folder).iterdir():
            text = file_path.read_text()
```

```
            embedding = embed(text)
            collection.add(
                documents=[text], embeddings=[embedding], ids=[file_path.name],
            )
        return collection

define:
    def ask(collection, question: str) -> None:
        question_embedding = embed(question)
        results = collection.query(
            query_embeddings=[question_embedding], n_results=3,
        )
        print the question, then results["ids"][0] and results["documents"][0]

test:
    collection = build_vector_store("docs")
    for question in your 3 questions:
        ask(collection, question)
```

Write `build_vector_store`'s loop and `ask` yourself, then compare against the Solution.

Once that's working, make it safe to run more than once, and honest about weak matches:

```
define:
    SCORE_CUTOFF = 1.2      # a distance worse than this means "not relevant"

    def build_vector_store(doc_folder):
        client = chromadb.Client()
        collection = client.get_or_create_collection(name="knowledge_base")
        if collection.count() == len(files in doc_folder):
            return collection   # already built
        for each file: embed and add, same as before
        return collection

    def search(collection, question, k=3, cutoff=SCORE_CUTOFF):
        question_embedding = embed(question)
        results = collection.query(
            query_embeddings=[question_embedding], n_results=k,
        )
        keep only results whose distance <= cutoff
        if nothing survives the cutoff: return [] and note "no relevant match"
        return the surviving (id, text, distance) tuples
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# knowledge_base_search_practice.py
SCORE_CUTOFF: float = 1.2

def build_vector_store(doc_folder: str):
    chroma_client = chromadb.Client()
    collection = chroma_client.get_or_create_collection(name="knowledge_base")
    # your turn: if this collection is already populated
    # (collection.count() > 0),
    # just return it instead of re-embedding and re-adding every file again
    ...
    return collection

def search(
    collection, question: str, k: int = 3, cutoff: float = SCORE_CUTOFF,
) -> list[dict]:
    question_embedding = embed(question)
    results = collection.query(
        query_embeddings=[question_embedding], n_results=k
    )
    # your turn: build a list of {"id":..., "text":..., "score":...} dicts,
    # but only for results whose distance is <= cutoff -- drop the rest
    ...
```

Then the clean rebuild, and the LangChain version with scores:

```
define:
    def rebuild_collection(name):
        try: chroma_client.delete_collection(name=name)
        except NotFoundError: pass        # first run, nothing to delete
        return chroma_client.create_collection(name=name)

    build_vector_store(doc_folder):
        documents = an empty dict
        for each path in Path(doc_folder).glob("*"):
            skip it if path.suffix is not ".txt" or ".md"
            documents[path.name] = path.read_text()
        collection = rebuild_collection("knowledge_base")
        one embed call for all texts, one collection.add(...) call

    LangChain:
        old = Chroma(collection_name="knowledge_base",
                     embedding_function=..., persist_directory="./chroma_db")
        old.delete_collection()
        store = Chroma.from_texts(texts, embeddings, metadatas=...,
                                  collection_name="knowledge_base",
                                  persist_directory="./chroma_db")
```

```
        for document, score in store.similarity_search_with_score(q, k=3):
            print score and document.metadata["source_file"]
```

Fill in the "already built" check and the cutoff filter yourself, then compare all of your finished versions against the Solution.

Full solution: Show me the solution